

Part 12: Kernel Density Estimation

The discussion of maximum likelihood concerns the problem of fitting **parametric** models to data. In other words, a particular parametric form is assumed for the probability model (e.g., normal, exponential, etc.) and the parameters of those models are estimated from data using the MLE.

One could view fitting parametric models as **parametric density estimation** in the case that the data is best modelled using a continuous distribution.

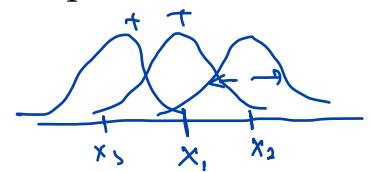
Nonparametric density estimation refers to techniques that estimate the **distribution** for a variable, without assuming any parametric form. We can view this operation as a “smoothing” of the observed data, done in a manner that allows the data to define the shape of the density.

Histograms are a simple example of a nonparametric density estimator. While these are useful, they suffer from limitations, namely the arbitrariness of the binning, and discontinuous (“jagged”) nature of the estimate of a distribution that is typically assumed to be smooth. Smooth estimators also have statistical efficiency advantages over histograms.

The standard (smooth) nonparametric approach to density estimation is the **kernel density estimator**. This estimate is constructed by summing a smooth **kernel function** centered at each of the observed data points.

Formally, we write:

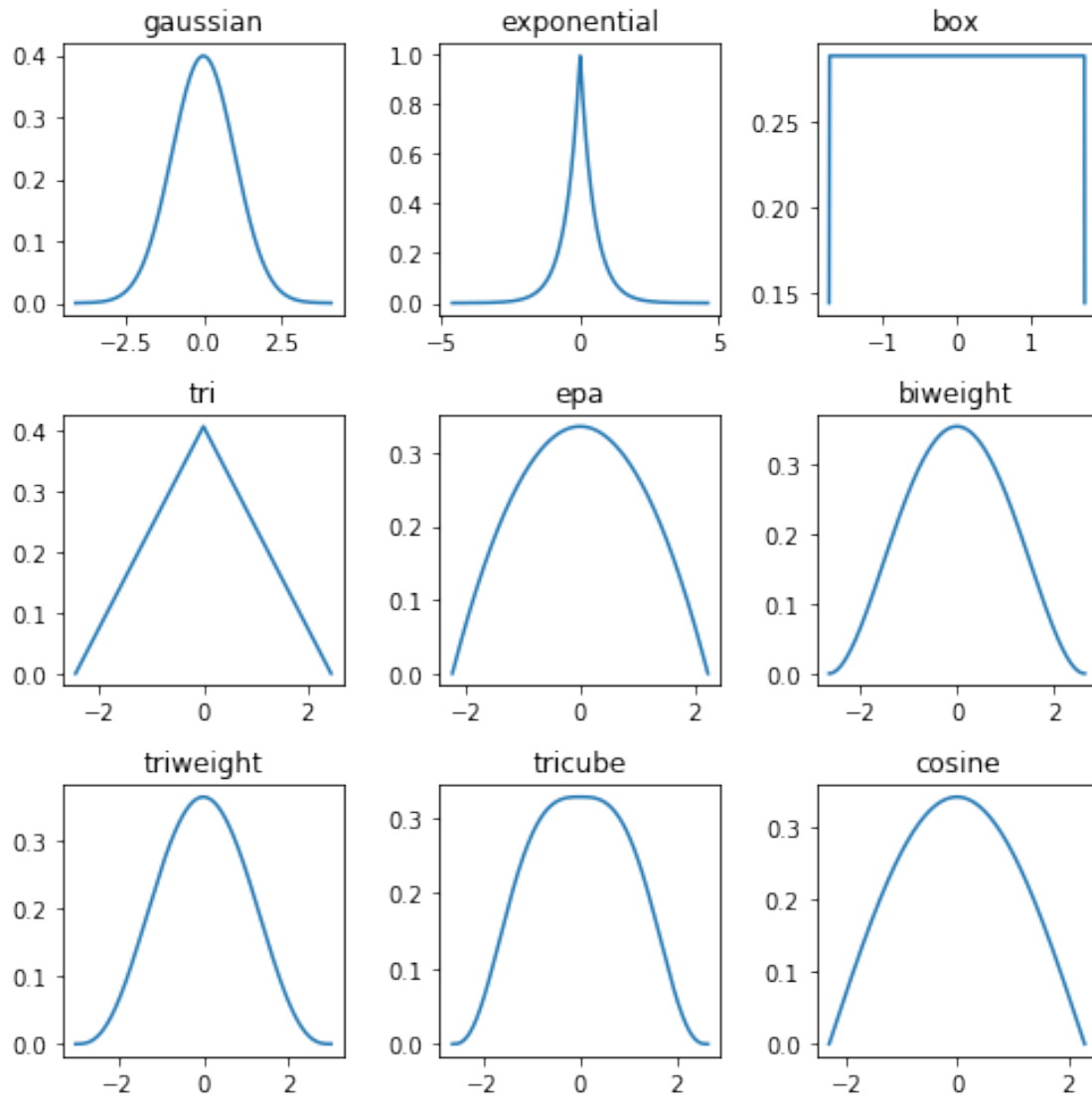
$$\hat{f}_h(x) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right)$$



where $K(\cdot)$ is the **kernel function** and h is the **smoothing parameter** or **bandwidth**. The sample is $x_1, x_2, \dots, x_n$.

The kernel function is itself a density, almost always taken to be a smooth density peaked at zero, and symmetric around zero.

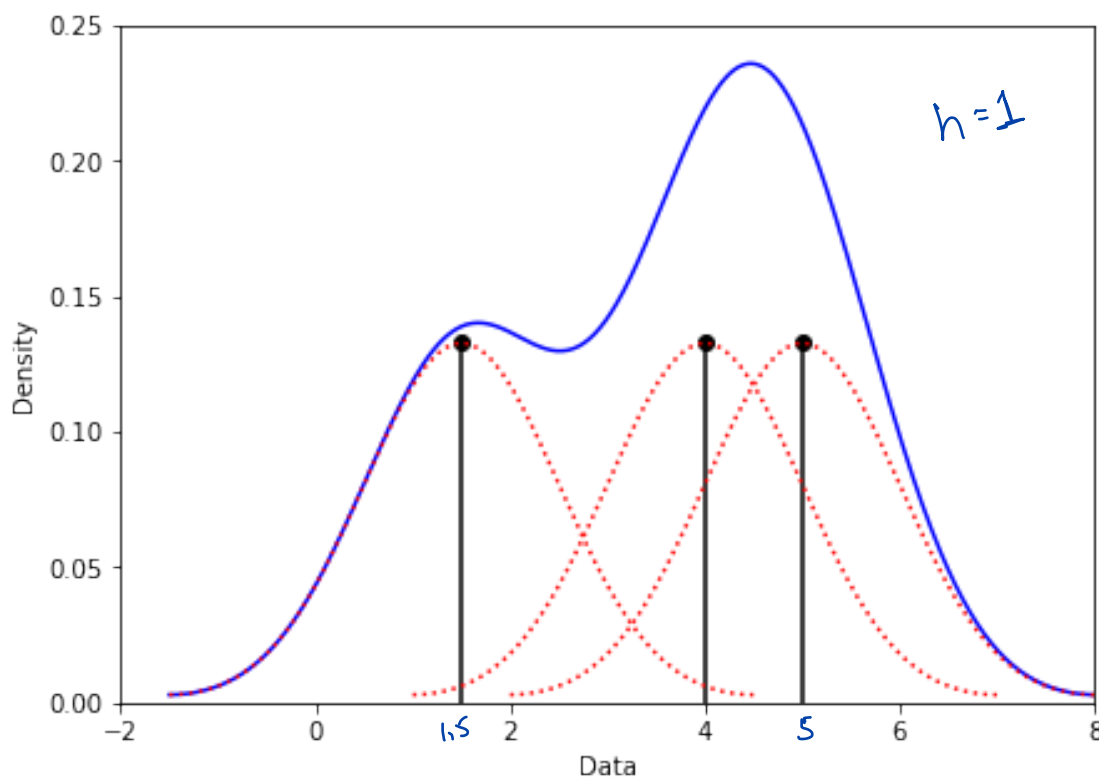
The package `KDEpy` incorporates nine different choices for the kernel functions, as shown below.



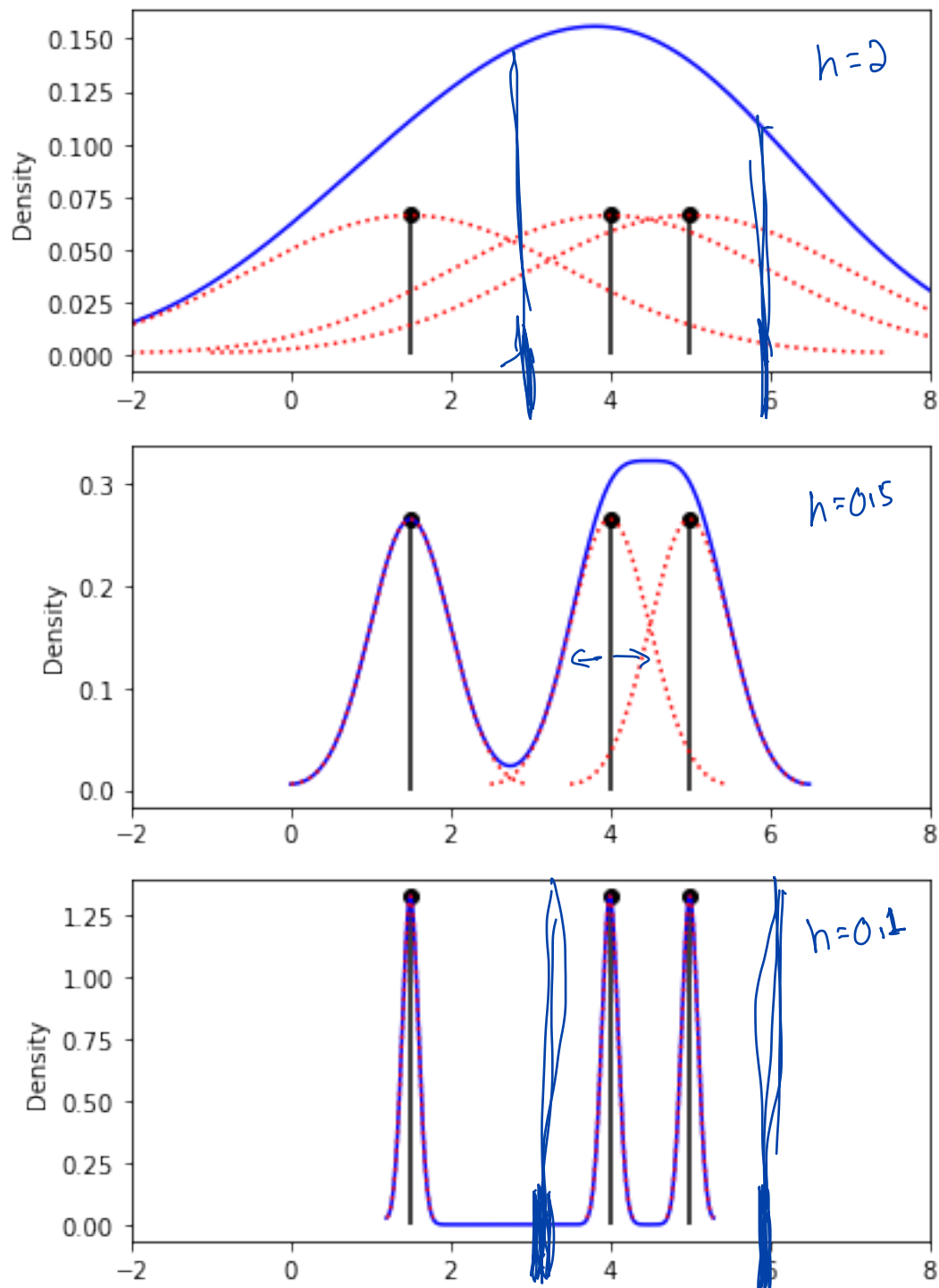
The choice of the kernel is not too important: Any smooth kernel should do an adequate job of creating a useful density estimate. The bandwidth h is influential, however.

Larger values of h result in a density estimate that is smoother. Smaller values of h produce an estimate that is “rough” and “wiggly.”

As a very simple example to illustrate the concept, imagine my data set consisted of three observations: 1.5, 4, and 5. The figure below illustrates the construction of the kernel density estimator in the case where $h = 1$.



The cases where $h = 2$, $h = 0.5$, and $h = 0.1$, respectively:



Exercise: Discuss the effect of varying h . How does it relate to the bias/variance tradeoff?

Recall
$$\text{MSE} = \text{variance} + \text{bias}^2$$

As h is chosen smaller, get estimator with larger variance (a lot of variability), but it has small bias (it will catch any features, real or not, in the training sample)

The opposite occurs when h chosen large: small variance, but large bias.

Hope to find optimal h to balance these two extremes.

$$\text{MSE}(\hat{f}_h(x))$$
 

Exercise: What is the relationship between using a Gaussian kernel and making an assumption that a sample is drawn from a Gaussian (normal) distribution?

NONE. The kernel is simply a shape for the "bump" placed at each obs.

Rigorous ways of choosing h do exist. See Jones, Marron, and Sheather (1996) for an overview.

Briefly, the theory is built on the assumption that the available sample $x_1, x_2, \dots, x_n$ are drawn i.i.d. from some population with density $f(\cdot)$. The criterion that one seeks to minimize is the **mean integrated squared error**:

$$\text{MISE} = E \left[\int (\hat{f}_h(x) - f(x))^2 dx \right].$$

As is often the case, consideration is made of the **asymptotic** behavior of the criterion as n increases. One can show that the **asymptotic mean integrated squared error** is

$$\text{AMISE} = \underbrace{\frac{R(K)}{nh}}_{\text{variance}} + \underbrace{h^4 R(f'') \left(\frac{1}{2} \int x^2 K(x) dx \right)^2}_{\text{bias}} \approx \text{MISE}$$

where

$$R(\phi) = \int \phi^2(x) dx$$

The optimal choice for h can then be shown to be

$$h = \left[\frac{R(K)}{n R(f'') (x^2 K(x))^2} \right]^{1/5}$$

The **Sheather-Jones (S-J)** approach to finding the bandwidth is to find the h which solves

$$h = \left[\frac{R(K)}{n R(\hat{f}_{g(h)}'') (x^2 K(x))^2} \right]^{1/5}$$

where $g(h)$ is some function of h which accounts for the fact that the optimal choice of bandwidth for estimating f'' is not the same as the optimal choice for estimating f .

The choice of h can also be dictated by less theoretical considerations.

In particular, if the motivation is to achieve a summary of a distribution via **smoothing** the distribution to a particular **scale**.

One could, for example, smooth a distribution using multiple bandwidth choices, and then using the resulting smooth estimates as input into a model. Formal procedures could then be used to determine how useful each “scale” is to the problem at hand, i.e., a prediction challenge.

Kernel Density Estimation in Python

We will discuss the use of `KDEpy` for the purpose of kernel density estimation. More information can be found at <https://kdepy.readthedocs.io/en/latest/index.html>.

This package includes a function `FFTKDE()` which implements a fast fourier transform-based algorithm for calculating the kernel density estimate, providing significant speedup. (See <https://kdepy.readthedocs.io/en/latest/introduction.html#Speed> for a discussion.)

The argument `kernel` specifies the kernel function; the default is `"gaussian"`.

The bandwidth is provided using the argument `bw`. The user can specify either a number, or specify a method for determining the bandwidth: `bw="ISJ"` calculates an “improved” version of the S-J bandwidth, while `scott` and `silverman` are two **normal-reference** bandwidth selection approaches, meaning that they are optimized to work well when the underlying distribution is normal.

Exercise: Try the following code. Consider what happens as the sample size and the bandwidth selection method are varied.

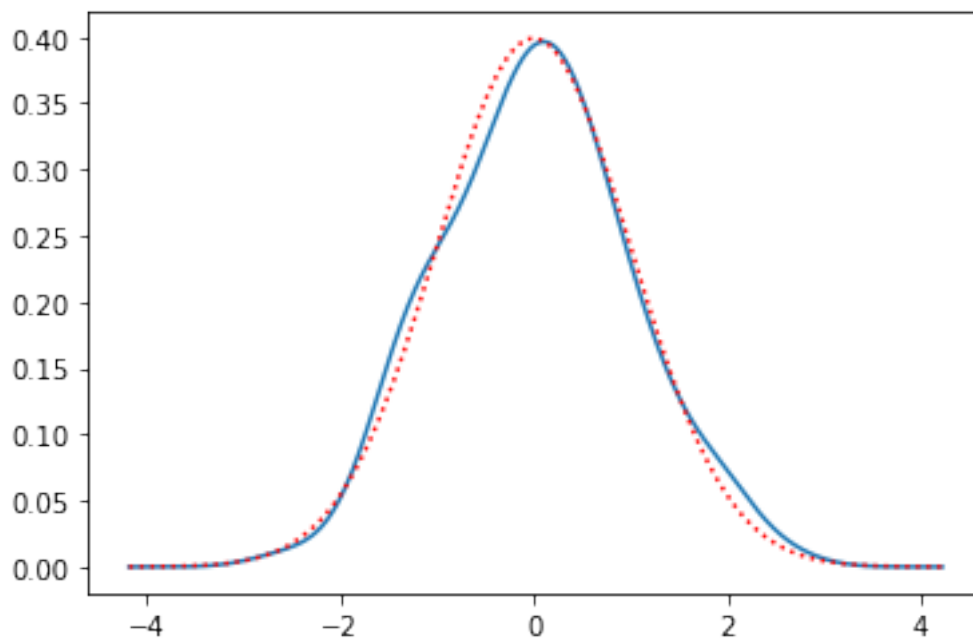
```
In [4]: from scipy.stats import norm

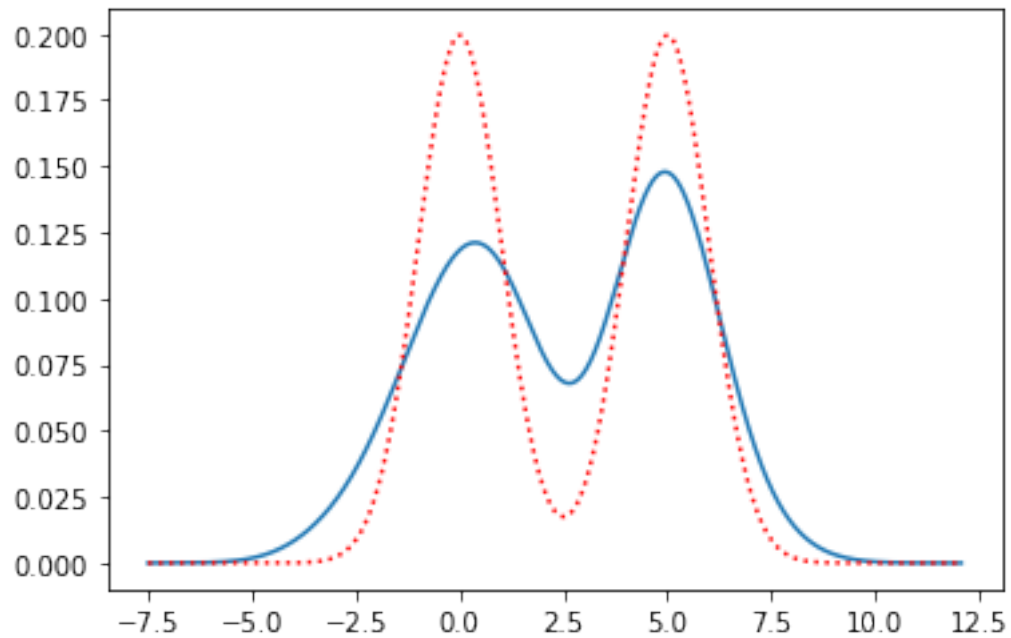
n = 100
data = norm.rvs(size=n)
x,y = FFTKDE(kernel='gaussian', bw='silverman').\
    fit(data).evaluate()

plt.plot(x, y)

plt.plot(x, norm.pdf(x), color="red",
         linestyle="dotted")
plt.show()
```

```
data2 = [norm.rvs(size=round(n/2)),  
         norm.rvs(5,size=round(n/2))]  
x,y = FFKDE(kernel='gaussian', bw='silverman').\  
    fit(data2).evaluate()  
  
plt.plot(x, y)  
  
plt.plot(x, 0.5*norm.pdf(x)+0.5*norm.pdf(x,5),  
         color="red", linestyle="dotted")  
plt.show()
```





The package 'seaborn' includes a function 'distplot' that allows for easy plotting of the kernel density estimate.

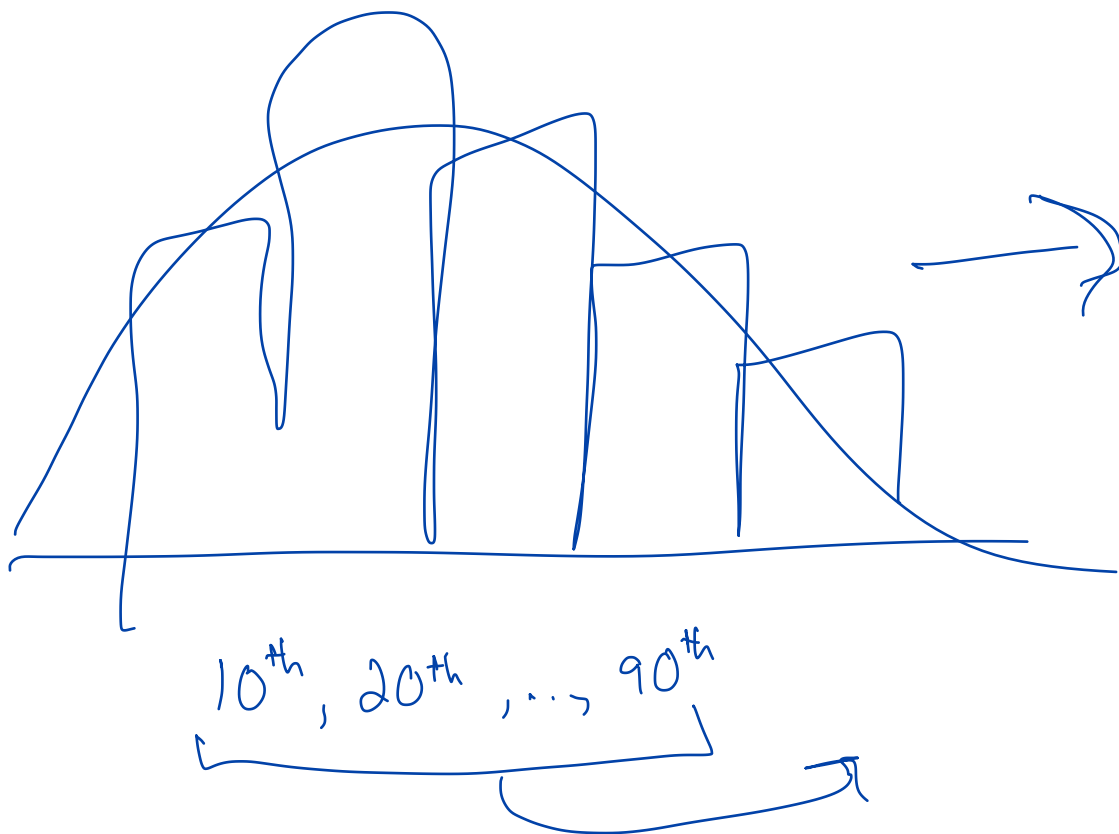
Here, we obtain data on Kelloggs from 2010 to 2016, and calculate the log daily returns over that period.

```
In [5]: import yfinance as yf
```

```
    Kdat = yf.download("K", start="2010-01-01",  
                      end="2018-12-31")
```

```
    ldrK = np.log(Kdat['Adj Close']).diff()[1:]
```

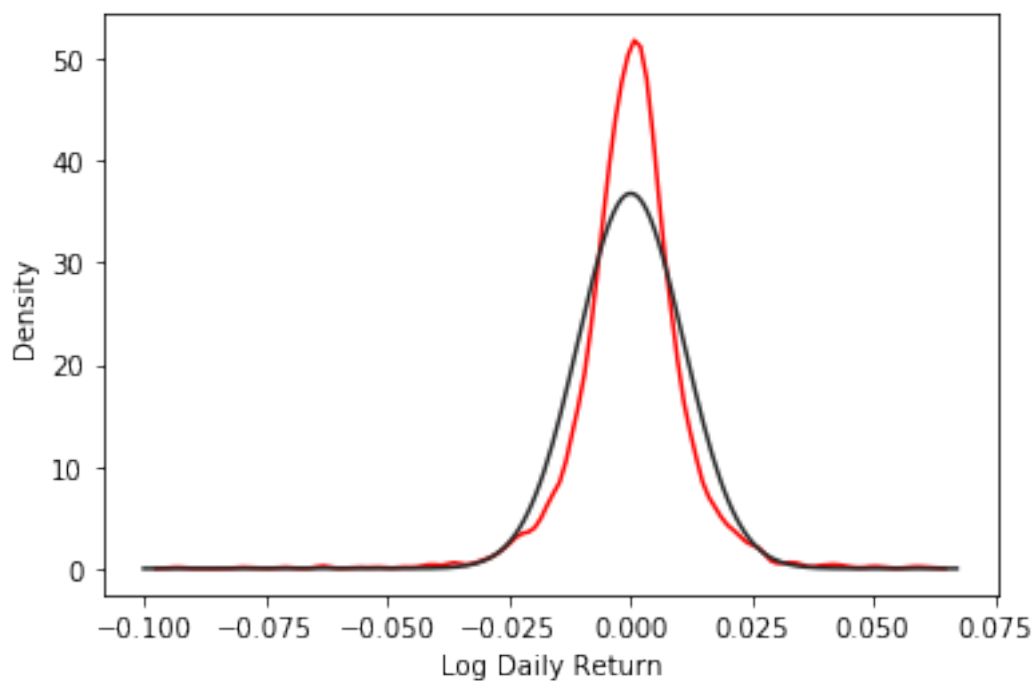
```
[*****100%*****] 1 of 1 down
```



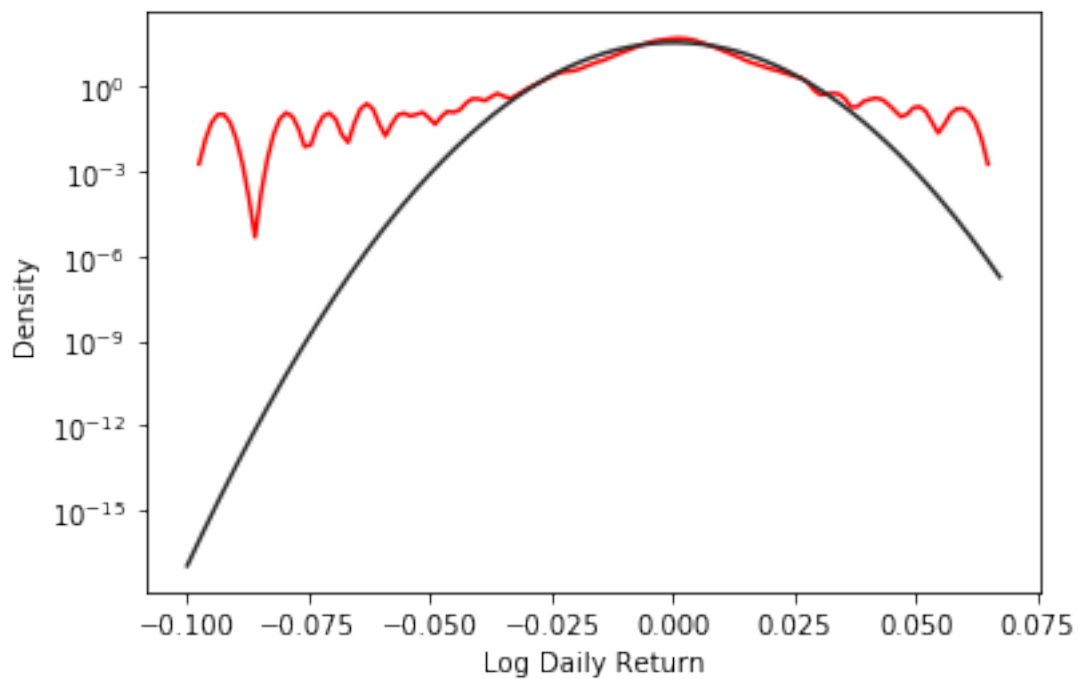
Next, construct the kernel density estimate plot. Note that by setting `fit=norm`, we are able to see the best-fitting normal curve.

```
In [6]: import seaborn as sns
```

```
ax = sns.distplot(ldrK, hist=False,  
                  kde_kws={"bw": "silverman"},  
                  color="red", fit=norm)  
plt.xlabel("Log Daily Return")  
plt.ylabel("Density")  
plt.show()
```



The comparison on the log scale is much more informative.



Exercise: This example illustrates a common use for kernel density estimation, namely to assess the validity of an assumed parametric form. What conclusion can you draw in this case?

Converting to the CDF and Percentiles

The package `statsmodels` includes a function `KDEUnivariate()` which, while is not as fast or flexible as the function provided by `KDEpy`, it does enable easy calculation of the **cumulative distribution function (CDF)** and its inverse (i.e., percentiles) derived from the smooth kernel density estimate.

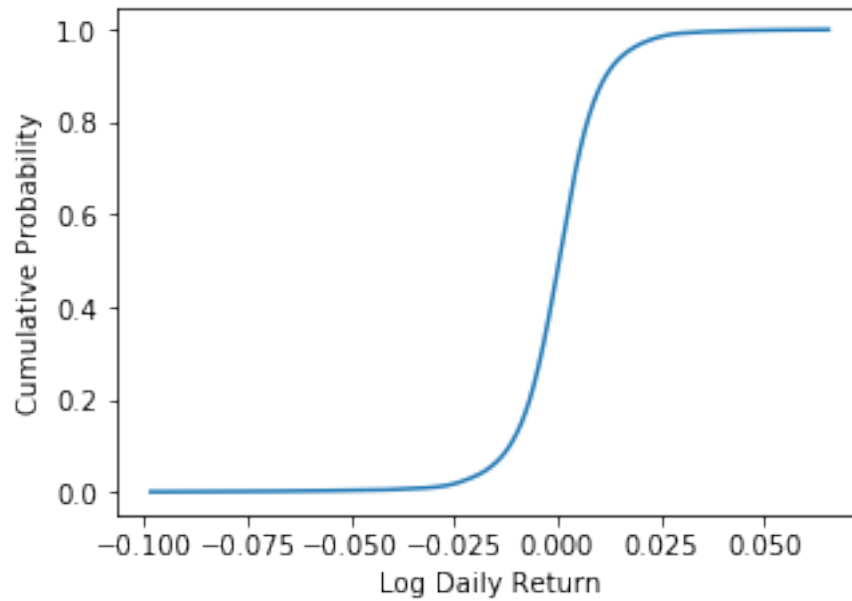
The default is to use the Gaussian kernel, which enables the use of faster calculation via FFT. The other available kernels are listed here, and can be specified to via the `kernel` argument to `.fit()`.

```
In [8]: from statsmodels.nonparametric.kde \
        import kernel_switch
        list(kernel_switch.keys())
```

```
Out[8]: ['gau', 'epa', 'uni', 'tri', 'biw', 'triw', 'cos', 'cos
```

```
In [9]: import statsmodels.api as sm
        dens = sm.nonparametric.KDEUnivariate(ldrK)
        dens.fit()

        fig = plt.figure(figsize=(5, 3.5), frameon=False)
        plt.plot(dens.support, dens.cdf)
        plt.xlabel("Log Daily Return")
        plt.ylabel("Cumulative Probability")
        plt.show()
```



It is possible to evaluate the CDF at any value using the following syntax.

```
In [10]: from scipy.interpolate import interp1d

         cdffunc = interp1d(dens.support, dens.cdf)

         cdffunc(0)
```

```
Out[10]: array(0.47858166)
```